

第7章 文件操作

讨论

- 你接触过哪些文件？
 - 电脑上是什么样？
 - 手机上是什么样？
- 遇到最多的文件类型是？
- 最独特的文件类型是？

文件分类

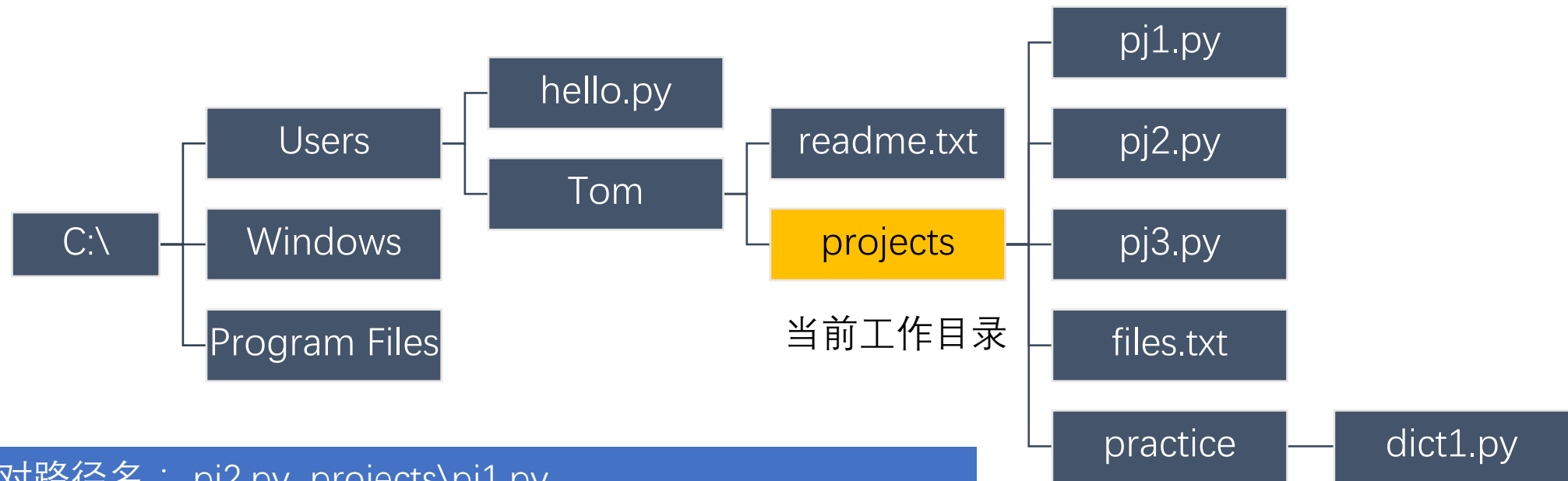
- 按照数据的组织形式，文件可分为文本文件和二进制文件
- 文本文件
 - 存储的是常规字符串，由若干文本行组成，通常每行以换行符'\n'结尾。
 - 常规字符串是指记事本或其他文本编辑器能正常显示、编辑并且人类能够直接阅读和理解的字符串，如英文字母、汉字、数字字符串。
 - 文本文件里面的字符采用某种编码方式（'utf-8 '、）编码后保存
 - 常用文本文件工具：notepad++，ultraedit，notepad，typora，linux下的gedit等。
- 二进制文件
 - 存储的是一个一个的字节，无法用常规的文本处理软件读写
 - 包含：数据库文件、图像文件、可执行文件、音视频文件、Office文档等等。
 - 需要使用专门的软件进行解码后读取、显示、修改或执行。
 - 常用的二进制文件工具：ultraedit、ultrahex、notepad++(安装hexedit插件)、FrhedPortable

文件和目录

- 文件以目录（文件夹）的方式组织成目录树
 - 一个目录可包含多个文件，也可以包含子目录，这些子目录再包含文件或子目录
- 每个文件或目录可以通过目录树中从根节点开始，按照树的结构找到其子节点，如此继续直到到达指定的位置
 - 在Linux和MacOS系统中只有一棵目录树，而微软操作系统中对于每个驱动器维护一棵目录树(C:; D:....)
 - 路径名(pathname): 从起点开始直到结束位置途中经过的节点名字组合在一起，之间通过**路径分隔符**隔开
 - Linux和MacOS系统中为 '/', 而微软系统中为 '\\'
 - 在实践中都可以使用 / 分割路径名，python会转换为操作系统所采用的路径分隔符
- **绝对路径名**: 以目录树中的根开始的路径名

绝对路径名: C:\Users\student
\AppData\Roaming\Python\Python36\site-packages

```
>>> import os
>>> os.sep
'\\'
>>> os.altsep
 '/'
```



- **相对路径名**：不以目录树中的根开始的路径名,引入**工作目录**的概念，相对路径名从工作目录开始

假设当前工作目录为projects:

- **files.txt** 表示projects下的files.txt
- **practice/dict1.py** 表示practice子目录下的dict1.py
- 特殊的目录名：.(一个点)表示当前目录，..(两个点)表示当前目录的父目录
- ../readme.txt 表示projects的父目录(这里为c:\users\Tom)中的readme.txt
- ../../hello.py 表示c:\users\hello.py
- 绝对路径：你的应用程序移动到任何位置都可以访问，但是要考虑到复制到另外一台机器，是否还能够通过绝对路径访问。
- **相对路径**：相关的文件可以**整体移动**到另外一个位置、另外一台机器，还能够访问

目录操作：当前工作目录（课堂练习 10Min）

```
import os
```

```
os.getcwd() #获得当前工作目录
```

```
os.mkdir(os.getcwd() + '\\temp') #创建目录
```

```
os.chdir(os.getcwd() + '\\temp') #改变当前工作目录
```

```
os.getcwd()
```

```
os.mkdir('test')
```

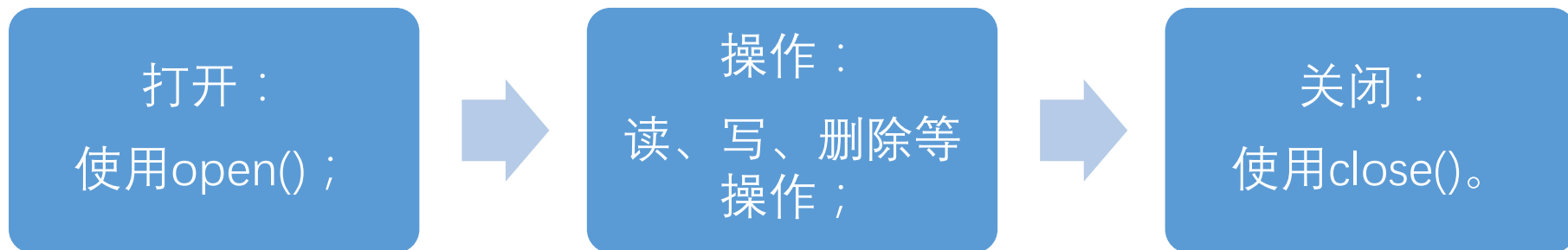
```
os.listdir('.') #列出目录
```

```
os.rmdir('test' ) #删除目录
```

#尝试通过Python将工作目录切换至D盘（或者桌面），创建并进入Python目录，创建Lesson1-Lesson10，共10个目录，回到Python目录，列出所有内容，最后删除今天新建的内容（注意不要误删有用文件）（10分钟）

7.1 文件对象

- 文件对象是python内置对象之一
 - 是一种资源；
 - 需要打开文件对象；
 - 同时，需要释放（关闭）文件对象。
- 文件对象的操作遵循特定的流程：三步



文件操作步骤

建立通道>>

单向通道

```
fobj = open(文件名, 打开方式)
```

文件作为**数据来源**：
`fobj = open(文件名, 'r')`

文件作为**数据目的**：
`fobj = open(文件名, 'w')` #重写
或：
`fobj = open(文件名, 'a')` #追加

- `fobj.read()`
- `fobj.readlines()`
- `fobj.readline()`

- `fobj.write()`
- `fobj.writelines()`

关闭通道>>

```
fobj.close()
```

文件打开详解

- 文件读写的第一步是打开文件

open(file, mode='r', buffering=-1, encoding=None, errors=None)以指定模式打开相应的文件，返回某种类型的文件对象

文件对象名 = open (文件名[, 访问模式])

- 文件名file: 建议采用raw string，不需要转义\
- 访问模式：打开文件后的处理方式；
 - rwxa必须指定一个，且只能指定一个
 - tb+(红色标记值) 可选，缺省为t，即文本方式打开，
 - 如果不指定mode，默认为 'r' ，即打开文本文件读
- 编码方式：在文本模式时给出了字符采用的编码
 - 缺省为locale.getpreferredencoding()获得的locale编码
 - 如果为'cp936'，表示GBK
- 缓冲：默认值是缓存模式。
 - 0表示不缓存，1表示缓存，大于1则表示缓冲区的大小，小于0表示使用默认的缓冲区大小
- errors: 文本文件读写时编码或解码出现错误时怎样处理

值	说明
r	读模式
w	写模式，不存在时创建,文件存在时 截取覆盖原有文件！！！！
x	写模式，先创建文件然后打开写;文件存在时抛异常FileExistsError
a	附加模式，不存在时会创建，存在时会附加到文件尾部
t	文本模式
b	二进制模式
+	读写模式，较少使用！！可使用r+ w+ x+ a+，注意w会清空原有文件的内容

文件打开典型调用方法

- 典型调用方式

- 以读模式打开文件

- `f = open('a.txt' )`

- 以读模式打开文本文件， 编码方式为'utf-8'

- `f = open('hello.py', encoding='utf-8')`

- 以写模式打开文件

- `f = open('b.txt','w')`

- 以附加模式打开文件

- `f = open('c.txt','a')`

- 以读写附加模式打开文件（文件不存在则创建文件， 否则置文件指针到文件尾）：

- `f = open('b.csv','a+')`

- 以读模式打开二进制文件

- `f = open('c.dat','rb')`

练习：寻找各种文件，看看二进制文件的样子（8min）

```
f = open("1.gif", "rb")
try:
    byte = f.read(1)
    while byte != "":
        print(byte)
        input("Press Enter to continue...")
        byte = f.read(1)
finally:
    f.close()
```

文件对象

- 文件对象在_io模块中实现

```
>>> import _io
>>> dir(_io)
>>> dir(_io.TextIOWrapper)
```

- 这些不同类型的文件对象提供的接口类似,但具体实现有所区别

```
>>> f = open('sample.txt')
>>> f
<_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp936'>
>>> fw = open('sample.txt','wb')
>>> fw
<_io.BufferedWriter name='sample.txt'>
>>> fr = open('sample.txt','rb')
>>> fr
<_io.BufferedReader name='sample.txt'>
```

- 文件对象属性

属性	描述
name	文件名
mode	打开模式
encoding	编码方式, 仅对文本方式有效
errors	编解码出错时的处理方式
closed	是否已经关闭

```
def print_file_property(file='sample.txt'):
    fh = open(file)
    print('file name   : %s' % fh.name)
    print('access mode: %s' % fh.mode)
    print('encoding    : %s' % fh.encoding)
    print('closed      : %s' % fh.closed)
    fh.close()

if __name__ == '__main__':
    #setup() #在备注中
    print_file_property()
```

file_property.py

```
file name   : sample.txt
access mode: r
encoding    : cp936
closed      : False
```

文件操作模式

文件打开之后一定要关闭，释放相应的资源，在以写方式打开时，关闭还保证写的内容同步到硬盘上

• 模式一

```
handler = open("sample.txt")
lines = handler.readlines()
handler.close()
```

```
print(lines)
```

• 优缺点

- 最常见
- 如有异常，行不正常，释放。

• 模式二

```
lines=[]
handler=open("sample.txt")
try:
```

在实际开发中，读写文件应优先考虑使用上下文管理语句with，关键字with可以自动管理资源，不论因为什么原因（哪怕是代码引发了异常）跳出with块，**总能保证文件被正确关闭**，

方式；
证资源的释放。

• 模式三

```
lines=[]
with open("sample.txt") as handler:
    lines = handler.readlines()
```

```
print(lines)
```

• 优缺点

- 比较安全的操作方式；
- 可以自动进行资源管理。

异常处理

先试试下面代码

```
datafile = open(input())  
print('encoding : %s' % datafile.encoding)  
datafile.close()
```

异常 (exception)

- 简单地说，异常是指程序运行时引发的错误。引发错误的原因有很多，例如名字错误、类型错误、除零、下标越界、文件不存在、网络异常、字典键错误、磁盘空间不足，等等。
- 当Python检测到一个错误时，解释器就会指出当前代码已无法继续执行下去，这时候就出现了异常。如果这些异常得不到正确的处理将会导致程序终止运行。

```
>>> a = int(input("please input an integer:"))
please input an integer:2.a
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    a = int(input("please input an integer:"))
ValueError: invalid literal for int() with base 10: '2.a'
```

*严格来所，语法错误和逻辑错误不属于异常，但有些语法错误往往会导致异常，例如由于大小写拼写错误而访问不存在的对象，这需要要在编写程序的时候改正。

8.1 什么是异常 (cont.)

- 合理地使用异常处理结果可以使得程序更加健壮，具有更强的容错性，不会因为用户不小心的错误输入或其他运行时原因而造成程序终止。
- 也可以使用异常处理结构为用户提供更加友好的提示。
- 程序出现异常或错误之后是否能够调试程序并快速定位和解决存在的问题也是程序员综合水平和能力的重要体现方式之一。
- 异常分为两个阶段：第一个阶段是引起异常发生的错误；第二个阶段是检测并处理阶段。
- 不建议使用异常来代替常规的检查，如必要if...else判断。
- 应避免过多依赖于异常处理机制。
- 当程序出现错误，python会自动引发异常，也可以通过raise显式地引发异常。

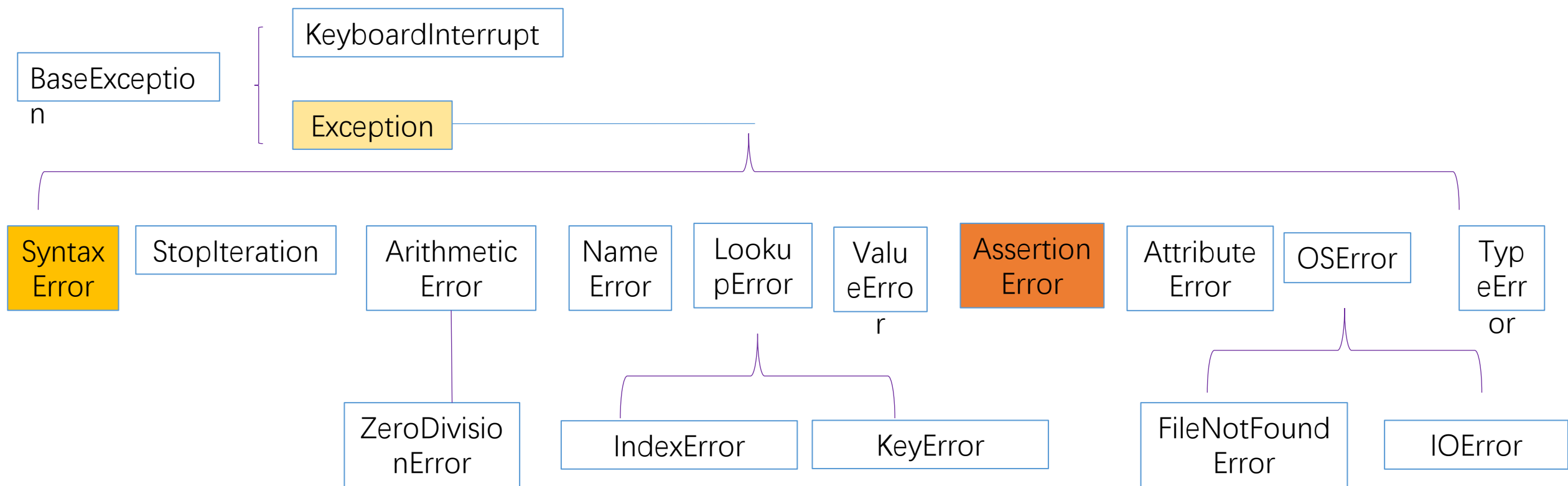
```
if type(score) is str:  
    raise ValueError("score 类型有错")
```

8.2 Python中的异常类

- 下面列出了常用的异常类

异常	描述
NameError	尝试访问一个没有声明的变量
ZeroDivisionError	除数为 0
SyntaxError	语法错误
IndexError	索引超出序列范围
KeyError	请求一个不存在的字典关键字
IOError	输入输出错误（例：读取不存在的文件）
AttributeError	尝试访问未知的对象属性
ValueError	传给函数的参数类型不正确（例：给 <code>int()</code> 函数传入字符串）
AssertionError	断言异常

8.2 Python中的异常类：部分类架构



抛出异常raise语句

- 主动抛出异常
 - 定义自己的异常类时；
 - 或者需要抛出异常时。
- raise语法 `raise [expression [from expression2]]`
 - expression：可以是一个异常类的实例对象；也可以是异常类，系统会首先创建一个该类的实例，然后抛出该异常对象；：
 - from expression2:给出了将两个异常连接起来的方法，表示异常是由于另一个expression2引起的（不作要求）。

```
raise Exception
raise Exception('foo occurred!')
if type(score) is str:
    raise ValueError("score 类型有错")
raise Exception('foo occurred!') from Exception('another error
occurred!')
```

raise例子

```
>>> int("12eef")|
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#4>", line 1, in <module>
```

```
    int("12eef")
```

```
ValueError: invalid literal for int() with base 10: '12eef'
```

```
>>> score="123"
```

```
>>> if type(score) is str:
```

```
    raise ValueError("score 类型有错")
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#7>", line 2, in <module>
```

```
    raise ValueError("score 类型有错")
```

```
ValueError: score 类型有错
```

自定义异常类 (不作要求)

- 下面的例子演示了自定义的异常类
 - 必须继承Exception类或者其子类：Exception为所有异常类的基类；

```
class MyError(Exception):  
    def __init__(self, value):  
        self.value = value  
    def __str__(self):  
        return repr("My error:%d"%self.value)
```

```
try:  
    raise MyError(2*2)  
except MyError as e:  
    print('My exception occurred', e)  
    raise
```

```
My exception occurred 'My error:4'  
Traceback (most recent call last):  
  File "<pyshell#4>", line 2, in <module>  
    raise MyError(2*2)  
MyError: 'My error:4'  
>>>
```

错误如何处理

**先尝试，处理异常继续进行
没有异常**

**try, except
else**

语法规则

```
try:  
    <语句> #运行别的代码  
except <名字> :  
    <语句> #如果在try部份引发了'name'异常  
except <名字>, <数据>:  
    <语句> #如果引发了'name'异常，获得附加的数据  
else:  
    <语句> #如果没有异常发生
```

```
try:
    datafile = open(input())

except IOError:
    print("Error: 没有找到文件或读取文件失败")
else:
    print('encoding : %s' % datafile.encoding)

    datafile.close()
```

```
try:
    datafile = open(input())

except:
    print("Error: 没有找到文件或读取文件失败")
else:
    print('encoding : %s' % datafile.encoding)

    datafile.close()
```

错误如何处理

先尝试，处理异常

最终

try, except

finally

finally

```
try:  
<语句>  
finally:  
<语句> #退出try时总会执行
```

```
try:
    datafile = open(input())

except IOError:
    print("Error: 没有找到文件或读取文件失败")
else:
    print('encoding : %s' % datafile.encoding)

    datafile.close()

finally:
    print("处理完成")
```

8.3 Python中的异常处理结构

try:

<tryBody> #被监控的代码

except [expression [as identifier]]: # 至少一次, 在多次出现时不带表达式except应该是最后一个。
按序匹配找到对应的exception为止

<exceptionBody> #异常处理代码

except # 可选的

except # 可选的

else: # 可选的, 没有异常出现时执行

<elseBody>

finally: # 可选的, 不管异常有没有都要执行

<finallyBody>

- expression应该为

- 异常类

- 或者异常类的元组, 表示其中任一异常出现

8.3 Python中的异常处理结构

- 第一种方式没有except块
 - 执行<tryBody>代码
 - 不管有没有异常都会执行<finallyBody>块

```
try:
    <tryBody> #被监控的代码
finally:
    <finallyBody>
```

```
>>> try:
        print(A)
    finally:
        print("finally run!")
```

```
finally run!
Traceback (most recent call last):
  File "<pyshell#17>", line 2, in <module>
    print(A)
NameError: name 'A' is not defined
```

8.3.1 try.....except结构

- 这是最基本的处理结构；
- 具有两种形式。

代码处理形式一

```
try:
    ...
except BaseException:
except Exception: #使用异常基类，捕获所有的异常
    except_block #异常处理代码
```

代码处理形式二

```
try:
    try_block #被监控的代码
except Exception1: #使用特定异常类
    except_block #针对性异常处理代码
```

优势：能够处理所有的异常

- 建议1
 - 尽量显式捕捉可能会出现的异常，并编写具有针对性的代码；

try.....except : 示例1

```
>>> while True:
    try:
        x = int(input("Please enter a number: "))
        break
    except ValueError:
        print("That was no valid number. Try again...")
```

```
Please enter a number: a
That was no valid number. Try again...
Please enter a number: b
That was no valid number. Try again...
Please enter a number: c
That was no valid number. Try again...
Please enter a number: 10
```

try.....except : 示例2 (不作要求)

```
try:
    raise Exception('spam', 'eggs')
except Exception as inst:
    print(type(inst))      # the exception instance
    print(inst.args)      # arguments stored in .args
    print(inst)           # __str__ allows args to be printed directly,
                          # but may be overridden in exception subclasses
    x, y = inst.args      # unpack args
```

```
    print('x =', x)
    print('y =', y)
print(x)
print(inst)
```

```
<class 'Exception'>
('spam', 'eggs')
('spam', 'eggs')
x = spam
y = eggs
spam
```

traceback (most recent call last):
UnboundLocalError: local variable 'inst' referenced before
assignment

expression as **instance**

instance为异常对象

补充内容：在exception block并不是一个新的scope，但是instance在退出该block后不可用

8.3.2 try.....exceptelse : 示例1(1)

```
a_list = ['China', 'America', 'England', 'France']
while True:
    print('请输入字符串的序号')

    try:
        n = int(input())
        print(a_list[n])
    except IndexError:
        print('列表元素的下标越界，请重新输入字符串的序号')
    else:
        break
```

Copy&Run

- 分析上述的代码
 - 输入错误：例如输入的数值超出了正常的序号，会产生异常；执行红色 print 语句，并继续循环；
 - 没有异常时：输出列表中正确的字符串，并且退出循环。
 - 但是输入了非数值？ → 没有捕获异常，程序退出

8.3.3 帶有多個except的try結構

```
try:
    try_block           #被監控的語句
except Exception1:
    except_block_1      #處理異常1的語句
except Exception2:
    except_block_2      #處理異常2的語句
...
```

- 功能：
 - 根據實際產生的異常，執行其中的異常程序塊。
 - 按照順序匹配，一旦匹配執行異常程序塊，後面的except塊不再執行

帶有多個except的try結構 示例1(2)

```
a_list = ['China', 'America', 'England', 'France']
while True:
    print('請輸入字符串的序號')
    try:
        n = int(input())
        print(a_list[n])
    except IndexError:
        print('列表元素的下標越界，請重新輸入字符串的序號')
    except ValueError:
        print('輸入不是十進制整數，請重新輸入字符串的序號')
    else:
        break
```

带有多个except的try：示例2

- 典型例子：除法。

```
try:
    x=eval(input('请输入被除数: '))
    y=eval(input('请输入除数: '))
    z= x / y
except ZeroDivisionError:
    print('除数不能为零')
except TypeError:
    print('被除数和除数应为数值类型')
except NameError:
    print('变量不存在')
except Exception as e:
    print('其他错误:', e)
else:
    print(x, '/', y, '=', z)
```

- 建议2

- 最后一个except用来捕捉Exception (BaseException)。

帶有多個except的try：示例3

```
def parse_int(file='sample.txt'):
    try:
        f = open(file)
        s = f.readline()    #如果这一行执行时出错怎么办
        f.close()
        i = int(s.strip())
        print(i)
    except (OSError, ValueError) as inst:
        print(type(inst), inst)
    except:
        print("Unexpected error:", sys.exc_info()[0])
        raise
```

功能：打开文件，
读一行内容，将
其转换为整数，
并且输出

```
if __name__ == '__main__':
    setup()    # 创建测试环境，生成相应的文件
    try:
        parse_int('sample.txt')    #该文件的第一行为整数，正常运行输出结果
        parse_int('sample1.txt')    # 该文件不存在，匹配OSError
        parse_int('sample2.txt')    #该文件第一行为非整数，触发异常ValueError
        parse_int('sample3.txt')    # 该文件以utf-8编码，readline()出错，
                                     # 匹配ValueError，但文件并没有关闭

    except Exception as e:
        print(e)
```

不管是正常执行，
还是异常出现时都
希望能够关闭打开
的文件

8.3.4 try.....exceptfinally结构

- 特点

- Finally中的语句总会执行；
- 可以用于清理工作，以便释放资源。

- 典型结构如下

```
try:
    try_block                #被监控的代码
except:
    except_block             #例外处理程序块
finally:
    finally_block            #无论如何都会执行
```

```
>>> try:
        print(A)

except:
    print("capture the error:")
finally:
    print("finally run!")
```

```
capture the error:
finally run!
```

```
>>> try:
        print(A)

except Exception as e:
    print("capture the error:" ,e)
finally:
    print("finally run!")
```

```
capture the error: name 'A' is not defined
finally run!
```

```
>>> try:
        print(A)

except NameError as e:
    print("capture the error:" ,e)
finally:
    print("finally run!")
```

```
capture the error: name 'A' is not defined
finally run!
```

try.....exceptfinally : 示例1 (1)

- 典型例程：文件的读取。
- 仅仅能防止文件读取操作出现异常而导致文件不能正常关闭

```
try:
    f = open('sample2.txt', 'r')
    line = f.readline()
    print(line)
finally:
    print("finally running")
    f.close()
```

```
>>> try:
      f = open('sample2.txt', 'r')
      line = f.readline()
      print(line)
    finally:
      print("finally running")
      f.close()
```

```
finally running
Traceback (most recent call last):
  File "<pyshell#1>", line 2, in <module>
    f = open('sample2.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'sample2.txt'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<pyshell#1>", line 7, in <module>
    f.close()
NameError: name 'f' is not defined
>>>
```

- 如果文件没有创建，则在finally中会产生异常。

8.3.4 try.....exceptfinally结构: 示例1

```
def parse_int2(file='sample.txt'):
    try:
        f = open(file)
        s = f.readline()
        # f.close()
        i = int(s.strip())
        print(i)
    except (OSError, ValueError) as inst:
        print(type(inst), inst)
    except:
        print("Unexpected error:",
sys.exc_info()[0])
        raise
    finally:
        f.close()
```

如果open时异常?
f未定义, 从而抛出
NameError



```
def parse_int3(file='sample.txt'):
    f = None
    try:
        f = open(file)
        s = f.readline()
        # f.close()
        i = int(s.strip())
        print(i)
    except (OSError, ValueError) as inst:
        print(type(inst), inst)
    except:
        print("Unexpected error:",
sys.exc_info()[0])
        raise
    finally:
        if f:
            f.close()
```

```
try:
    with open(file) as f:
        s = f.readline()
        i = int(s.strip())
        print(i)
except (OSError, ValueError) as inst:
    print(type(inst), inst)
except:
    print("Unexpected error:", sys.exc_info()[0])
    raise
```

建议采用with block打开文件, 不用管文件是否关闭的问题

try.....exceptfinally : 示例2

- 如果try子句中的异常没有被捕获与处理，或者except子句、else子句中的代码出现了异常，那么这些异常将会在finally中的语句执行完后再次抛出。

```
def divide(x, y):  
    try:  
        result = x / y  
    except ZeroDivisionError:  
        print("division by zero!")  
    else:  
        print("result is", result)  
    finally:  
        print("executing finally clause")
```

```
>>> divide(2, 1)  
result is 2.0  
executing finally clause  
>>> divide(2, 0)  
division by zero!  
executing finally clause  
>>> divide("2", "1")  
executing finally clause  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
  File "<stdin>", line 3, in divide  
TypeError: unsupported operand type(s)  
for /: 'str' and 'str'
```

try.....exceptfinally : 示例3

- **finally代码中：应该尽量避免使用return语句**
- 在finally字句中有return或者break时会取消要抛出的异常！

```
def demo_div(a, b):  
    try:  
        return a/b  
    except ZeroDivisionError:  
        pass  
    finally:  
        return -1
```

```
>>> demo_div(1, 0)  
-1  
>>> demo_div("1","2") #虽然有异常，但是被取消了  
-1  
>>> demo_div(1, 2)  
-1  
>>> demo_div(10, 2)  
-1
```

异常处理总结

- 可以通过raise子句来抛出异常
- except子句用于捕获指定的异常，按照顺序匹配，一旦匹配，后面的except子句跳过
- except子句可以通过 as instance来获得异常对象，但是注意except子句退出后instance不再有效
- 可选的else子句在没有异常时执行
- 可选的finally子句不管有没有异常都会执行
- 如果没有捕获异常，则该异常会往try外层抛出
 - 在finally子句中有return或者break时会取消要抛出的异常！

8.4 断言与上下文处理

- 断言与上下文处理
 - 两种特殊的异常处理形式；
 - 形式上比通常的异常处理简单；

8.4.1 断言(不作要求)

```
if __debug__:
    if not expression:
        raise AssertionError
        # raise AssertionError(reason)
```

- 断言语句的语法

assert expression[, reason]

- 当判断表达式expression为真时，什么都不做；如果表达式为假，则抛出异常。
- 即要求expression为真,用于开发程序时对特定必须满足的条件进行验证
- 什么时候__debug__为True?
 - 目前的实现中，正常情况为True
 - 当Python脚本以-O选项编译为字节码文件时，__debug__为False, assert语句不会产生任何代码

8.4.2 上下文管理

- 使用with语句进行上下文管理，在with_block结束时会自动释放资源
- 避免了try...finally结构中的资源释放问题
- 不管异常是否出现都会释放在进入时获得的资源

```
with context_expr [as obj]:  
    with_block
```

用户也可以自己实现某个资源的上下文管理（不作要求）
依赖于python语言的magic method，需要实现

__enter__() 和 __exit__() 两个方法：上下文
管理协议；

或者通过引用contextlib，并使用
@contextlib.contextmanager方式实现。

with语句等价于如下代码

```
obj = context_expr  
obj.__enter__()  
try:  
    with_block  
finally:  
    obj.__exit__()
```

with语句使用：示例

1. 显示文件内容

```
def read_file_with(file='sample.txt'):
    with open(file) as f:
        for line in f:
            print(line,end='')
```

2. 拷贝文件file到file-bk

```
def copy_file_with2(file='sample.txt'):
    with open(file) as fr:
        with open(file+'-bk',"w") as fw:
            for line in fr:
                fw.write(line)
```

```
def copy_file_with(file='sample.txt'):
    with open(file) as fr, open(file+'-
bk',"w") as fw:
        for line in fr:
            fw.write(line)
```

二进制文件读写

二进制文件读写

- 二进制文件:

- 打开模式中 mode 中的 'b'

file = open('sample.dat', 'rb')

file = open('sample.dat', 'wb')

- write(bytes_obj) **传递的是bytes对象**, 将该对象中的数据写入文件, 返回写入的字节数

- read(size=-1) **返回的是bytes对象**

- 不传递参数或者size<0时表示从文件中读直到文件结束EOF (end of file)
 - size>0从当前位置读指定数量的字节或文件结束EOF为止
 - 如果读时已经EOF返回**空bytes对象**

```
def write_binary_file(file='sample.dat'):  
    f=open(file, 'wb')  
    chars = bytes(range(256))  
    f.write(chars)  
    chars = bytes(range(255, -1, -1))  
    f.write(chars)  
    f.close()
```

```
def read_binary_file(file='sample.dat'):  
    f=open(file, 'rb')  
    s = f.read()  
    print(s)  
    f.close()
```

bin_demo.py

Copy&Run

文件对象的常用操作

文件打开后会维护一个文件指针位置，给出了下一次的读写操作时所对应的位置。以a模式打开时，指针为文件的当前结束位置，否则为0

	方法	说明
读	f.read([size=-1])	读取最大size个字符(字节)或EOF， size为-1时表示文件中所有字符(字节)
	f.readline()	读取字符直到遇到换行符或者EOF， 如果读时马上遇到EOF， 则返回空字符串。 注意在 读时遇到\n时， \n也包括在内
	f.readlines()	读取直到EOF， 返回一个列表， 每个元素为读到的每行内容(包括行尾换行符\n)
	next(f)	以读方式打开的文件是一个迭代器， 下一个元素为文件中下一行的内容(包括\n)， 直到EOF为止
写	f.write((text))	将字符串(字节串)text写入文件， 返回写入的字符(字节)个数
	f.writelines(lines)	把lines中的每个元素逐个写入文件， 要求list的元素必须为字符串， 返回None
其他	f.flush()	缓冲区的内容同步到存储设备
	f.close()	关闭文件， 释放资源， 写模式下还同步缓冲区内容到存储设备
	f.truncate([size])	文件截取后长度变为size， 缺省为当前文件指针位置(文件打开时初始为0)， 返回文件的当前长度
	f.tell()	访问打开文件的当前指针位置
	f.seek(offset, whence=0)	移动文件指针位置到基准位置的offset， 缺省whence=0， 表示从头部开始， whence=1,表示当前指针位置, whence=2表示文件结尾开始

7.2 文本文件操作案例精选：write

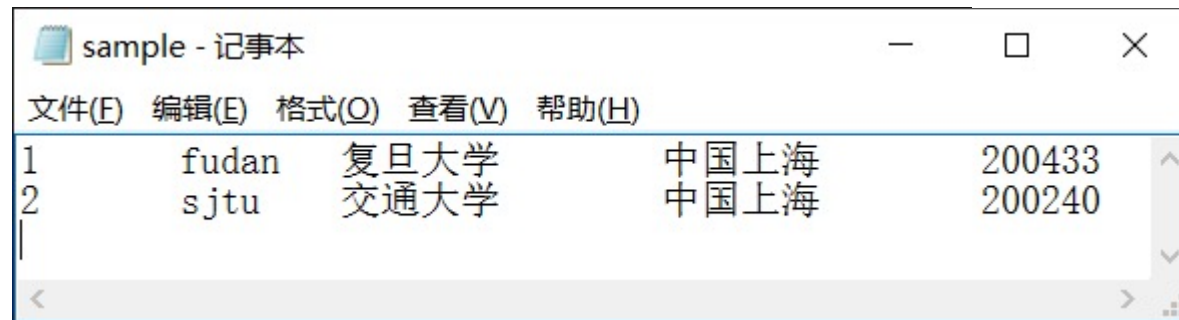
- 例7-1 向文本文件写入内容。
 - 可以仿照模式一和模式三写代码如右侧；
 - 注意s1和s2中都有行结束标记。
- 文本文件：
 - 打开写可以采用模式w、a和x，可选的+表示同时也打开读模式，如果不指定编码(encoding)，缺省为[`locale.getpreferredencoding\(\)`](#)
 - `write(str_obj)`: 将字符串的内容str_obj按照打开文件时编码方式转换为字节串后写入文件

```
def write_file(file='sample.txt'):  
    f=open(file,'w')  
    s1='1\tfudan\t复旦大学\t中国上海\t200433\n'  
    s2='2\tsjtu\t交通大学\t中国上海\t200240\n'  
    f.write(s1)  
    f.write(s2)  
    f.close()
```

```
def write_file_2(file='sample.txt'):  
    s1='3\tfudan\t复旦大学\t中国上海\t200433\n'  
    s2='4\tsjtu\t交通大学\t中国上海\t200240\n'  
    with open('sample.txt','a') as f:  
        f.write(s1)  
        f.write(s2)
```

txt_demo.py

Copy&Run



扩展知识：换行符

- 尽管不同操作系统使用的行分隔符可能各不一样，但是python解释器在进行**文本文件的读写**时统一用**\n(new line)**作为行分隔符，会自动进行相应的转换
- 为什么有回车换行？早期英文打字机上：
 - 回车符(Carriage Return, \r): 回到carriage开始的位置（最左边）
 - 换行符(Line Feed, \n): 往下移动一行

换行符

- 系统缺省的行分隔符
 - Windows系统为'\r\n'
 - Linux系统为'\n'
 - MacOS为'\r'

```
>>> import os
```

```
>>> os.linesep
```

```
'\r\n'
```

- os.linesep给出了系统缺省的行之间的分隔符，可能是 \r \n \r\n等。
- python一般采用Unix模式 \n
- python做了一些自动转换的工作，在window读取时将\r\n转换为\n，写的时候\n转换为\r\n

文本文件写：writelines

- writelines(lines)：传递的参数为一个可迭代对象，其中的元素为**字符串**，表示将按照顺序将所有的字符串按照文件打开时设置的编码方式编码后写入文件
- 虽然名字叫writelines，但可迭代对象的元素写入时**并不会自动添加换行符**

```
def write_lines(lines):  
    for line in lines:  
        f.write(line)
```

- 为确保内容已经从缓存中写入文件，可调用flush函数

```
def write_file_writelines(file='sample.txt'):  
    f=open(file,'w')  
    s = ['1\tfudan\t复旦大学\t中国上海\t200433\n',  
        '2\ttsjtu\t交通大学\t中国上海\t200240\n']  
    f.writelines(s)  
    f.flush()  
    f.close()
```

文本文件读：read

- 打开文本文件读，
 - 模式为'rt'或者其他读写模式（如'w+'），文本模式(t)可省略
 - 如果不指定编码(encoding)，缺省为[locale.getpreferredencoding\(\)](#)
- read(size=-1)：从文件中读取size个**字符**（注意不是字节，而是unicode字符），如果size小于0则表示读取之后的所有内容
- seek(offset,whence=0)（**不作要求**）
 - 可以将文件指针移动到文件的某个位置（偏移量）
 - 缺省为从文件开始处算起的第几个**字节**（注意不是字符）
 - whence=0表示从文件开始，whence=1表示当前位置,whence=2表示文件结尾

```
def read_file_read(file='sample.txt'):  
    with open(file, 'r') as f:  
        firstN = f.read(10)  
        f.seek(0)  
        lines = f.read()  
    print('first 10 chars:\n%s' % firstN)  
    print('-'*40)  
    print(lines)
```

txt_demo.py

Copy&Run

文本文件读：readline和readlines

- `readline()`: 从文件读数据直到换行或者文件结束，返回读到的字符串，换行也包括在内。刚开始读遇到EOF时返回空字符串
- `readlines()`: 从文件读数据直到文件结束，返回一个字符串列表，列表中的元素为读到的每行字符串(包括换行符)

```
>>> write_file()
>>> read_file_readline()
1      fudan    复旦大学      中国上海      200433
2      sjtu     交通大学      中国上海      200240
>>> read_file_readlines()
['1\tfudan\t复旦大学\t中国上海\t200433\n', '2\tsjtu\t交通大学\t中国上海\t200240\n']

1      fudan    复旦大学      中国上海      200433
2      sjtu     交通大学      中国上海      200240
>>> |
```

```
def read_file_readline(file='sample.txt'):
    f = open(file, 'r')
    while True:
        line = f.readline()
        if not line:
            break
        print(line, end='')
    f.close()
```

```
def read_file_readlines(file='sample.txt'):
    f = open(file, 'r')
    lines = f.readlines()
    f.close()
    print(lines)
    print()
    for line in lines:
        print(line, end='')
```

文件对象支持iterator协议

- file支持iterator协议，意味着调用next(file)获得下一行，即相当于file.readline()

```
f = open(filename)
while True:
    line = f.readline()
    if not line:
        break
    process(line)
f.close()
```

```
f = open(filename)
for line in f:
    process(line)
f.close()
```

```
lines = list(open('somefile.txt'))
# 相当于调用
open('somefile.txt').readlines()
```

文件操作案例

文本文件内容操作例子：

- **1** 向文本文件中写入内容，然后再读出。

```
s = 'Hello world\n文本文件的读取方法\n文本文件的写入方法\n'
```

```
with open('sample.txt', 'w') as fp:    #默认使用cp936编码
    fp.write(s)
```

```
with open('sample.txt') as fp:         #默认使用cp936编码
    print(fp.read())
```

文本文件内容操作例子

- **2** 将一个CP936编码格式的文本文件中的内容全部复制到另一个使用UTF8编码的文本文件中。

```
def fileCopy(src, dst, srcEncoding, dstEncoding):  
    with open(src, 'r', encoding=srcEncoding) as srcfp:  
        with open(dst, 'w', encoding=dstEncoding) as dstfp:  
            dstfp.write(srcfp.read())
```

```
fileCopy('sample.txt', 'sample_new.txt', 'cp936', 'utf8')
```

文本文件内容操作例子

- **3** 遍历并输出文本文件的所有行内容。

```
with open('sample.txt') as fp:           #假设文件采用CP936编码
    for line in fp:                       #文件对象可以直接迭代
        print(line)
```

文本文件内容操作例子

- 4 统计文本文件中最长行的长度和该行的内容。

```
with open('sample.txt') as fp:
    result = [0, '']
    for line in fp:
        t = len(line)
        if t > result[0]:
            result = [t, line]
print(result)
```

文件操作系列：遍历目录

```
import os  
with os.scandir('my_directory') as entries:  
    for entry in entries:  
        print(entry.name)
```

文件操作系列：遍历文件

```
import os
```

```
basepath = 'my_directory'  
with os.scandir(basepath) as entries:  
    for entry in entries:  
        if entry.is_file():  
            print(entry.name)
```

```
for f_name in os.listdir('some_directory'):  
    if f_name.endswith('.txt'):  
        print(f_name)
```

创建目录

```
import os
```

```
os.mkdir('example_directory')
```

```
os.makedirs('2022/04/22')
```

遍历目录

```
for dirpath, dirnames, files in os.walk('.'):
    print(f'Found directory: {dirpath}')
    for file_name in files:
        print(file_name)
```

删除文件

```
import os
```

```
data_file = 'C:\\Users\\student\\Desktop\\data.txt'
```

```
os.remove(data_file)
```

复制文件、目录

```
import shutil
```

```
src = 'path/to/file.txt'  
dst = 'path/to/dest_dir'  
shutil.copy(src, dst)
```

```
import shutil  
dst = shutil.copytree('data_1', 'data1_backup')  
print(dst) # data1_backup
```

移动

- `import shutil`
- `dst = shutil.move('dir_1/', 'backup/')`
- `print(dst) # 'backup'`

重命名

```
import os
```

```
os.rename('first.zip', 'first_01.zip')
```

功能小结

- 获取文件属性
- 创建目录
- 文件名模式匹配
- 遍历目录树
- 创建临时文件和目录
- 删除文件和目录
- 复制、移动和重命名文件和目录
- 创建和解压ZIP和TAR档案
- 使用fileinput 模块打开多个文件